PROJECT REPORT

ON

“CampusConnect: Student Academic & Communication Portal”

INDIVIDUAL CONTRIBUTION REPORT

Submitted in partial fulfillment of the requirement for the award of Degree of

Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted by

AMAN KUMAR CHAUHAN

Enrollment No.: 24EARCS017

Individual responsibility: Authentication, Student/Teacher Profile, and Dashboard Shell

Under the guidance of

Er. Ram Babu Buri

Department of Computer Science & Engineering

Session: 2026–27

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO Industrial Area, Kukas, Jaipur

CERTIFICATE

This is to certify that the individual project report entitled “CampusConnect: Student Academic & Communication Portal — Authentication, User Profile and Dashboard Shell” has been prepared by Aman Kumar Chauhan (24EARCS017), student of Bachelor of Technology in Computer Science & Engineering at Arya College of Engineering & I.T., Jaipur, affiliated with Rajasthan Technical University, Kota, during the academic session 2026–27.

The report documents the student’s assigned contribution to the group project, with emphasis on the authentication foundation, student/teacher profile handling, and the shared dashboard shell. The report distinguishes work described in the available project history from features that remain planned or require final verification.

The work has been carried out under the guidance of Er. Ram Babu Buri. This report is submitted in partial fulfillment of the academic requirements of the programme.

Role	Name	Signature
Project Guide	Er. Ram Babu Buri	_____
Head of Department	Dr. Akhil Pandey	_____
Student	Aman Kumar Chauhan (24EARCS017)	_____

Date: _____ Place: Jaipur

Note: The final certificate wording and signature format should be confirmed with the department before printing.

DECLARATION

I, Aman Kumar Chauhan (24EARCS017), hereby declare that this individual report titled “CampusConnect: Student Academic & Communication Portal — Authentication, User Profile and Dashboard Shell” has been prepared to describe my assigned contribution to the CampusConnect group project.

The report has been written for academic submission and is based on the project context, implementation notes, and development workflow available at the time of preparation. It does not claim that every module proposed for the overall portal has been completed. Features that are not confirmed as working are explicitly identified as planned, partially implemented, or requiring verification.

I understand that source code, screenshots, test results, database schema, and the final application state should be checked against the current team repository before the report is submitted. I have not represented hypothetical test outcomes as measured results.

I declare that this report has not knowingly been copied from another student’s individual report and that external technologies and references are acknowledged in the references section.

Aman Kumar Chauhan

Enrollment No.: 24EARCS017

Department of Computer Science & Engineering

Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Er. Ram Babu Buri, for his guidance and academic support during the planning and development of CampusConnect. His supervision has helped me understand the importance of modular implementation, clear responsibilities, and systematic verification.

I am thankful to Dr. Akhil Pandey, Head of the Department of Computer Science & Engineering, Arya College of Engineering & I.T., Jaipur, for providing the academic environment in which this project is being developed. I also acknowledge the support of faculty members and the institution.

I would like to thank my CampusConnect teammates for coordinating work across the frontend, backend, database, and feature modules. Working in a shared GitHub repository has provided practical experience with branches, commits, integration, and resolving dependencies between modules.

I am especially grateful for the opportunity to work on authentication, student/teacher profile handling, and the common dashboard shell. These areas have helped me connect frontend routing and API calls with backend validation, persistence, and access control concepts.

Finally, I thank my family and friends for their encouragement and support throughout the project.

ABSTRACT

CampusConnect is a web-based Student Academic and Communication Portal intended to bring common academic and communication workflows into one institutional interface. The overall group project envisions access for students and teachers, with areas for academic information, announcements, clubs, chat, and profile management. The initial implementation is being developed for Arya College of Engineering & I.T., Jaipur.

This individual report focuses on the contribution assigned to Aman Kumar Chauhan: the authentication foundation, student/teacher profile area, and shared dashboard shell. The project uses React with JavaScript and Vite for the frontend, Java with Spring Boot and Maven for the backend, and MySQL for persistent data. Axios is used for frontend-to-backend HTTP communication. The recorded authentication work includes registration and login endpoints, role-specific identifiers, BCrypt password encoding, JWT token generation/validation, and frontend token handling; these details should be verified against the final branch before submission.

The dashboard contribution is limited to the common application shell, navigation, welcome area, profile access, logout entry point, routing, and placeholders for feature areas owned by other team members. The report does not claim ownership of the internal implementation of assignments, announcements, clubs, or chat.

This report explains requirements, design, implementation approach, security decisions, validation scenarios, known limitations, and future improvements. Any behavior not confirmed in the current application is labelled as planned or requiring final verification rather than being presented as completed.

TABLE OF CONTENTS

Section	Page
Certificate	2
Declaration	3
Acknowledgement	4
Abstract	5
Chapter 1 — Introduction	7
Chapter 2 — Project Scope and Individual Responsibility	8
Chapter 3 — Technology Stack and Development Environment	9
Chapter 4 — Requirements Analysis	10
Chapter 5 — System Architecture and Design	11
Chapter 6 — Authentication Module	12
Chapter 7 — Student and Teacher Profile Module	13
Chapter 8 — Dashboard Shell and Frontend Integration	14
Chapter 9 — Database and API Design	15
Chapter 10 — Security Architecture	16
Chapter 11 — Implementation Workflow and Version Control	17
Chapter 12 — Testing Strategy and Test Cases	18
Chapter 13 — Results, Limitations and Verification Status	19
Chapter 14 — Challenges and Learning Outcomes	20
Chapter 15 — Future Enhancements and Conclusion	21
References	22
Appendix A — Demo and Submission Checklist	23

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
BCrypt	Password hashing algorithm designed for password storage
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
JWT	JSON Web Token
MVC	Model–View–Controller
ORM	Object–Relational Mapping
REST	Representational State Transfer
RBAC	Role-Based Access Control
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
UX	User Experience
Vite	Frontend development server and build tool
XSS	Cross-Site Scripting

Terminology note: In this report, “implemented” means there is development history indicating that the functionality was written or exercised. It does not automatically mean the feature has passed a complete test suite or is production-ready. “Planned” means it is a proposed next step or an unverified capability.

CHAPTER 1 — INTRODUCTION

1.1 Background

College communities use different channels for academic notices, course-related communication, student information, and extracurricular coordination. When these activities are spread across disconnected tools, users may need to switch between platforms and information can become difficult to locate. CampusConnect proposes a common portal in which students and teachers can access role-appropriate institutional features through a consistent interface.

1.2 Problem Statement

The project addresses the need for a unified entry point for academic and communication services. The individual contribution addresses a foundational problem: before a portal can expose private profiles or role-specific pages, it needs a reliable way to identify users, validate credentials, maintain a session, and route users to the correct application area.

1.3 Objectives

- Provide a common registration and login experience for student and teacher users.
- Represent student and teacher roles and their distinct identifiers.
- Use password hashing rather than storing readable passwords.
- Connect the React interface to backend APIs and handle authentication responses.
- Provide a shared dashboard shell with navigation, profile access, and logout entry points.
- Keep other feature modules separate so teammates can implement them independently.

1.4 Individual Contribution

My assigned scope is Authentication, Student/Teacher Profile, and the shared Dashboard Shell. The report concentrates on this boundary and describes interfaces to other modules without claiming their internal implementation.

CHAPTER 2 — PROJECT SCOPE AND INDIVIDUAL RESPONSIBILITY

2.1 Overall Project Context

CampusConnect is a team project. The planned portal includes academic information, announcements, clubs/events, chat, user profiles, and a common dashboard. The team divides these areas so that each member can develop a module while using shared frontend conventions, backend endpoints, and database definitions.

2.2 Responsibility Matrix

Area	My responsibility	Status framing
Authentication	Registration/login flow, credential validation, token/session integration	Implementation recorded; final branch verification required
User profile	Student and teacher role/profile distinctions and access path	Core design assigned; full profile CRUD must be checked
Dashboard shell	Common layout, navigation, welcome/profile/logout entry points	Shell scope; feature pages may remain placeholders
Academic module	Interface/navigation only	Internal implementation owned by teammate
Announcements	Interface/navigation only	Internal implementation owned by teammate
Clubs and events	Interface/navigation only	Internal implementation owned by teammate
Chat	Interface/navigation only	Internal implementation owned by teammate

2.3 In-Scope Work

- Identity and role-aware entry into the application.
- Integration between React and backend authentication endpoints.
- Token storage/attachment and handling of unauthorized responses.
- Common dashboard structure and navigation placeholders.
- Documentation of validation scenarios and known limitations.

2.4 Out-of-Scope Work

This individual report does not claim responsibility for the complete academic workflow, announcement publishing, club management, group messaging, institution administration, or production deployment. Those are team-level or future features unless a current working implementation proves otherwise.

CHAPTER 3 — TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

Technology	Role in CampusConnect	Reason for use
React + JavaScript	Frontend UI and reusable components	Component-based interface and state-driven rendering
Vite	Frontend development/build tooling	Fast local development and bundling
React Router	Client-side navigation	Route transitions within the SPA
Tailwind CSS	Styling and responsive layout	Utility-first styling and consistent spacing
Axios	HTTP communication	Centralized API requests and interceptors
Java 21	Backend programming language	Object-oriented server-side implementation
Spring Boot	REST API and application framework	Dependency injection, controllers, configuration
Spring Data JPA / Hibernate	Persistence abstraction	Mapping Java entities to relational tables
Spring Security	Security integration foundation	Authentication and request protection where configured
MySQL	Relational database	Persistent structured records
Maven	Backend build/dependency management	Repeatable build and run commands
Git + GitHub	Source control and collaboration	Branches, commits, pull requests, integration

3.1 Development Setup

The recorded environment uses a Windows development machine, VS Code, Java 25 in the local setup history, Maven 3.10.0, Node.js/npm, and MySQL. The repository README specifies Java 21, Node 20+, Maven 3.9+, and MySQL 8+ as project targets. Local runtime versions should be aligned with the repository requirements before a final demonstration.

3.2 Run Sequence

1. Start MySQL and confirm the configured database exists.
2. Provide database and JWT settings through environment variables or the local configuration expected by the backend.
3. From the backend folder, run `mvn spring-boot:run`.
4. From the frontend folder, install dependencies if needed and run `npm run dev`.
5. Open the Vite local URL and verify that API base URL/CORS settings match the backend.

Secrets such as database passwords and JWT signing keys must not be committed to Git. The `.env.example` files should contain placeholders only.

CHAPTER 4 — REQUIREMENTS ANALYSIS

4.1 Functional Requirements

ID	Requirement	Priority
FR-01	A new user can submit registration details.	High
FR-02	The backend validates required fields and prevents duplicate email registration.	High
FR-03	Student and teacher registration use role-specific identifiers.	High
FR-04	A registered user can submit login credentials.	High
FR-05	Passwords are verified against stored password hashes.	High
FR-06	Successful login returns an authentication token/session response.	High
FR-07	Frontend requests can attach a Bearer token when available.	High
FR-08	Unauthorized responses can clear invalid local session data and route to login.	High
FR-09	A shared dashboard exposes navigation, profile access, and logout entry points.	Medium
FR-10	Full profile editing and feature-module content are available.	Planned / verify

4.2 Non-Functional Requirements

- Security: passwords must be hashed; secrets must be configured outside source control.
- Usability: form labels, validation feedback, and clear navigation should be provided.
- Maintainability: controllers, services, repositories, DTOs, and UI components should have separated responsibilities.
- Compatibility: frontend and backend must agree on endpoint paths, payload fields, and CORS origins.
- Reliability: errors should be handled without exposing sensitive internal details.
- Extensibility: new feature modules should connect through stable navigation and API contracts.

4.3 Users and Use Cases

Primary user roles are STUDENT and TEACHER. Both use the authentication entry point, but their profile attributes differ. A student may be identified by an enrollment number; a teacher may be identified by an employee ID. The precise list of stored fields must match the current entity definitions and database schema.

4.4 Constraints and Assumptions

The project is an academic prototype under active development. Availability of a local MySQL service, correct environment variables, matching frontend/backend ports, and compatible dependency versions are assumed. Role-based UI visibility is not a substitute for server-side authorization.

CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

5.1 Layered Architecture

Layer	Responsibility	Examples in project
Presentation	Renders forms, dashboard and feedback	React components, router, Tailwind CSS
API communication	Sends requests and handles responses	Axios client/interceptors
Controller	Maps HTTP routes to application operations	AuthController
Service	Applies validation and business rules	AuthService
Security	Loads user identity, verifies credentials/tokens	SecurityConfig, UserDetailsService, JwtService
Persistence	Reads/writes relational data	JPA repositories/entities
Database	Stores application records	MySQL schema

5.2 Request Flow

Step	Flow description
1	User submits login form in React.
2	Frontend sends credentials through Axios to the backend authentication endpoint.
3	Controller validates request shape and calls the service layer.
4	Service looks up the user and verifies the submitted password against the stored hash.
5	On success, token/session response is created; on failure, an appropriate error is returned.
6	Frontend handles the response, stores the token according to the current implementation, and navigates to the dashboard.
7	Subsequent API requests may attach the token; backend filters must validate it before protected resources are returned.

5.3 Design Principles

- Single responsibility for controller, service, persistence, and presentation layers.
- DTOs define API payloads without exposing internal entity structure unnecessarily.
- Role-aware behavior should be enforced on the server, not only hidden in the UI.
- Shared navigation should not imply that every linked feature is implemented.
- Environment-specific settings should remain configurable.

5.4 Architecture Boundary

The dashboard shell is a common frame. Academic, announcements, clubs/events, and chat may appear as navigation items or placeholders, but their business logic and data flows belong to the respective module owners. This boundary supports parallel development and reduces coupling.

CHAPTER 6 — AUTHENTICATION MODULE

6.1 Purpose

Authentication establishes the identity of a user before the application displays private or role-specific information. It is a prerequisite for profiles and protected dashboard features. The module accepts registration and login requests, validates the submitted information, and returns a response that the frontend can use to establish an authenticated state.

6.2 Registration Flow

6. The user chooses Student or Teacher and enters the required registration fields.
7. The frontend sends a request to `/api/auth/register`.
8. The controller receives a RegisterRequest DTO and delegates processing to AuthService.
9. The service validates required values and checks whether the email is already registered.
10. The role-specific identifier is validated according to the selected user type.
11. The password is encoded using BCrypt before persistence.
12. The service saves the user and returns a response that avoids exposing the password hash.

6.3 Login Flow

13. The user enters an email/username and password in the login interface.
14. The frontend sends a LoginRequest to `/api/auth/login`.
15. The backend locates the account and verifies credentials using the configured authentication/password encoder.
16. If authentication succeeds, the JWT service generates a signed token with an expiry.
17. The response returns the authentication result and token-related information expected by the frontend.
18. The frontend updates authentication state and navigates to the dashboard.

6.4 Relevant Backend Components

Component	Purpose
AuthController	Exposes registration and login HTTP routes.
RegisterRequest / LoginRequest	Carry validated request data.
AuthService	Coordinates validation, duplicate checks, password encoding, and token creation.
AuthResponse	Defines the response payload returned by authentication.
PasswordEncoder (BCrypt)	Hashes passwords for storage and verifies submitted passwords.
JwtService	Creates and validates signed JWTs and expiration.
CustomUserDetailsService	Loads user information for Spring Security.
SecurityConfig	Configures security filter chain, authentication manager, and request rules.

Verification note: component names above are based on the implementation notes available in the project history. Before final submission, confirm that each class is present and working on the latest `aman-auth-profile` branch.

CHAPTER 7 — STUDENT AND TEACHER PROFILE MODULE

7.1 Role-Aware Profile Model

CampusConnect distinguishes student and teacher users because their institutional identities and profile details are not identical. A student is commonly associated with an enrollment number and academic placement, while a teacher is associated with an employee identifier and professional department information. The exact persisted fields must be confirmed against the current schema rather than inferred from this conceptual model.

Profile type	Possible identifying / profile attributes	Verification status
Student	Enrollment number; course/programme; department; year; section	Conceptual requirements; confirm actual fields
Teacher	Employee ID; department; designation; office/room details	Conceptual requirements; confirm actual fields
Shared user	Name; email; password hash; role; account timestamps where implemented	Confirm entity/schema

7.2 Profile Responsibilities

- Keep shared login identity separate from role-specific profile information where the schema supports it.
- Use the authenticated identity to determine which profile may be returned.
- Validate editable profile fields on the server, not only in React forms.
- Avoid returning password hashes or sensitive security metadata in profile responses.
- Present only fields relevant to the signed-in role.

7.3 Profile Use Case

After successful authentication, a user opens Profile from the dashboard. The frontend should request the current user’s profile through an authenticated endpoint. The backend identifies the caller from the validated token and returns only that caller’s allowed profile data. When editing is implemented, the server should validate changes and persist them transactionally.

7.4 Implementation Status

Student/teacher role handling and role-specific registration identifiers are recorded in the development history. Full profile retrieval/edit endpoints and all listed fields should be labelled complete only after confirming the source code, database schema, and a successful UI/API test. This report therefore treats comprehensive profile CRUD as requiring verification unless demonstrated in the final build.

CHAPTER 8 — DASHBOARD SHELL AND FRONTEND INTEGRATION

8.1 Dashboard Shell Scope

The dashboard shell provides a consistent frame after login. Its purpose is navigation and shared layout rather than implementing every module shown in the navigation. The assigned shell includes a responsive sidebar/navbar, a welcome section, profile access, logout entry point, route structure, and placeholders or links for Academic, Announcements, Clubs, and Chat.

8.2 Authentication State

The frontend uses an authentication context to make the signed-in state available to components. The context is responsible for hydrating session state from the agreed storage mechanism, exposing the current user/token where applicable, and clearing session state during logout. The final code should be checked to confirm the exact context fields and persistence behavior.

8.3 Axios Interceptors

- Request interceptor: attach `Authorization: Bearer <token>` when a token is available and the request requires authentication.
- Response interceptor: handle 401 Unauthorized responses consistently, clear invalid session state, and route the user to login where appropriate.
- Avoid infinite redirect loops for login/register endpoints.
- Do not log tokens, passwords, or sensitive response payloads in production.

8.4 Routing and Protected Pages

React Router separates public routes such as login/register from authenticated routes such as the dashboard and profile. A protected-route component may prevent unauthenticated users from entering private UI routes. This client-side guard improves navigation but does not replace server-side authorization.

8.5 User Experience Considerations

- Show clear loading states while authentication requests are pending.
- Display helpful validation messages without exposing internal server exceptions.
- Keep the navigation responsive on smaller screens.
- Provide a visible logout action and avoid leaving stale private content visible after logout.
- Mark incomplete modules as placeholders rather than presenting them as functional.

CHAPTER 9 — DATABASE AND API DESIGN

9.1 Data Persistence

MySQL is the relational data store for the application. The backend uses Java persistence components to map application objects to database tables. The database schema file in the repository should be treated as the authority for actual table and column names. This report describes the logical data needed by authentication and profiles without inventing unverified SQL definitions.

9.2 Logical Entities

Entity concept	Purpose	Key considerations
User / account	Shared identity and login credentials	Unique email; encoded password; role; no plain-text password
Student profile	Student-specific institutional details	Enrollment number uniqueness if required; valid academic attributes
Teacher profile	Teacher-specific institutional details	Employee ID uniqueness if required; valid department/designation
Authentication response	Data returned after successful login	Token and safe user summary; no credential/hash disclosure

9.3 Known Authentication Endpoints

Method	Path	Purpose	Evidence status
POST	/api/auth/register	Create account	Recorded in controller implementation notes
POST	/api/auth/login	Authenticate account	Recorded in controller implementation notes
GET/PUT	Profile endpoints	Read/update current profile	Confirm actual route mappings before claiming

9.4 API Contract Rules

- Use request DTOs to define required input fields.
- Return suitable HTTP status codes for validation, conflicts, and authentication failures.
- Keep response structure consistent across success and error cases.
- Never include the password hash in any API response.
- Document the actual endpoint paths and JSON fields from source code before final printing.

9.5 Configuration

Database URL, database username/password, allowed frontend origins, and JWT signing secret/expiry are environment-specific. Local values should not be placed in a public repository. Use the example environment file as a template and keep the actual secret-bearing file ignored by Git.

CHAPTER 10 — SECURITY ARCHITECTURE

10.1 Password Storage

Passwords should be stored as one-way hashes using BCrypt, not as readable text. During registration, the raw password is passed to the encoder and only the resulting hash is persisted. During login, the supplied password is checked against the saved hash using the password verification function. A hash is not a value that should be returned to the frontend.

10.2 JWT Handling

A JSON Web Token can represent a signed authentication claim with an expiration time. The server must verify the token signature and expiry before trusting its claims. Tokens should be sent over HTTPS in a deployed environment and should not be written to logs or shared through screenshots. The current local implementation records JWT generation and validation; its final filter-chain behavior must be tested against protected routes.

10.3 Authorization

Authentication answers “who is the user?” Authorization answers “what may this user do?” Student/teacher role distinctions must be checked at backend endpoints when data access depends on role. Hiding a navigation item in React is not a security boundary.

10.4 CORS and Secrets

- Allow only known development/deployment origins rather than using a broad wildcard with credentials.
- Keep JWT signing keys and database credentials out of committed source code.
- Use different secrets for development and production.
- Validate request fields and avoid exposing stack traces in user-facing errors.
- Use dependency updates and least-privilege database credentials.

10.5 Threats and Mitigations

Threat	Mitigation approach
Credential disclosure	BCrypt hashing; no secrets in source control; safe logs
Token theft	Short expiry, secure transport, careful storage, logout cleanup
Role bypass	Server-side authorization checks on protected endpoints
Duplicate accounts	Unique constraints and service-level duplicate checks
Malformed input	DTO validation and consistent error handling
Cross-origin misuse	Explicit CORS allowlist and appropriate credential policy

CHAPTER 11 — IMPLEMENTATION WORKFLOW AND VERSION CONTROL

11.1 Branch-Based Development

The recorded working branch for this contribution is `aman-auth-profile`. Branch-based development isolates changes from the shared main branch and allows review before integration. The project has used GitHub pull requests and merged contributions from teammates, so preserving a clean working tree and pulling recent changes are important.

11.2 Typical Workflow

19. Pull the latest team branch and inspect the current repository status.
20. Create or switch to the assigned feature branch.
21. Implement a small, coherent change and run relevant checks locally.
22. Review the diff to ensure credentials, generated files, or unrelated edits are not included.
23. Commit with a message that describes the change.
24. Push the branch and open/update a pull request for review.
25. Resolve integration conflicts and retest after merging.

11.3 Integration Points

- Frontend request field names must match backend DTO properties.
- Frontend API base URL must match the running backend port.
- CORS configuration must allow the frontend development origin.
- Authentication response keys must match AuthContext and Axios interceptor expectations.
- Role strings must be consistent across database, backend, and UI.
- Dashboard routes must align with the pages actually present in the repository.

11.4 Local Run and Troubleshooting

Symptom	Likely cause / check
Backend exits on startup	Read the first exception above BUILD FAILURE; check Java/Maven compatibility and application properties.
Database connection refused	Confirm MySQL service is running, port/database name are correct, and credentials are valid.
Frontend request fails	Check API base URL, CORS origin, backend status, and browser network panel.
Login returns 401	Check account exists, password hash verification, payload keys, and authentication provider.
Dashboard redirects to login	Check token storage key, response shape, interceptor behavior, and protected-route logic.

CHAPTER 12 — TESTING STRATEGY AND TEST CASES

12.1 Testing Approach

Testing should cover validation at the UI, API, service, security, and database layers. The cases below are a test plan and expected behavior; they are not presented as executed results unless the team records evidence from the final application.

ID	Scenario	Expected result	Status
TC-01	Register with valid student data	Account is created; password is hashed	To execute/record
TC-02	Register with valid teacher data	Account is created with teacher identifier	To execute/record
TC-03	Register with an existing email	Request rejected with conflict/validation feedback	To execute/record
TC-04	Register with missing required field	Request rejected with clear validation response	To execute/record
TC-05	Login with correct credentials	Successful response and token issued	Login flow recorded; retest
TC-06	Login with incorrect password	Authentication denied without sensitive detail	To execute/record
TC-07	Call protected endpoint without token	Request rejected	To execute/record
TC-08	Call protected endpoint with expired token	Request rejected; frontend handles 401	To execute/record
TC-09	Open dashboard after login	Shared dashboard shell renders	Observed in prior development; retest
TC-10	Logout then open protected page	Session cleared and user returned to login	To execute/record
TC-11	Attempt role-inappropriate profile access	Backend denies unauthorized access	To execute/record
TC-12	Restart app and inspect configured persistence	Database records remain; session behavior matches design	To execute/record

12.2 Evidence to Capture

- Registration success and duplicate-email validation.
- Login success and invalid-credential response.
- Dashboard shown after successful login.
- Browser Network tab showing safe request/response fields without exposing tokens.
- Database row showing a password hash rather than the original password.
- Unauthorized request response and logout behavior.

12.3 Acceptance Criteria

The contribution is ready for demonstration when a student and teacher account can be created with valid role identifiers, duplicate and invalid data are rejected, valid credentials permit login, invalid credentials are denied, protected requests validate tokens, the dashboard shell renders after login, and logout clears the current session. All criteria should be checked against the actual current build.

CHAPTER 13 — RESULTS, LIMITATIONS AND VERIFICATION STATUS

13.1 Development Outcome

The project history records that the backend authentication controller/service, password encoding, JWT service, security configuration, and frontend token integration were developed. It also records a successful login form submission that redirected to the dashboard during local testing. These are useful development milestones, but they do not establish that every edge case or protected endpoint has passed a full regression suite.

13.2 Status Classification

Capability	Report classification	What to confirm
Register/login endpoints	Implemented in recorded development history	Run both endpoints on latest branch
BCrypt password encoding	Implementation recorded	Inspect stored DB value and verify login
JWT generation/validation	Implementation recorded	Test valid, expired, malformed, and missing token
Axios token interceptor	Implementation recorded	Check exact storage key and request headers
401 handling	Implementation recorded	Test expiry and ensure no redirect loop
Student/teacher role identifiers	Part of assigned design and service logic	Confirm entity/schema and validation
Full profile CRUD	Not confirmed in available notes	Inspect actual endpoints and UI; otherwise mark planned
Dashboard shell	Login-to-dashboard flow observed; shell scope assigned	Capture latest dashboard and verify logout
Announcements/academic/clubs/chat internals	Team/future modules outside individual scope	Do not claim as personally implemented

13.3 Limitations

- The application remains an academic prototype and should not be described as production-ready.
- A complete security review and automated test suite are not evidenced in the available project notes.
- Exact profile fields and database constraints require confirmation from `database/schema.sql` and current entity classes.
- Some dashboard destinations may be placeholders pending work by other team members.
- Runtime success can depend on local database, Java, Maven, Node, and environment configuration.

13.4 Responsible Reporting

The report deliberately avoids invented benchmark numbers, false test pass rates, or claims that planned modules are complete. Screenshots and final test outcomes should be added only after they are captured from the current build.

CHAPTER 14 — CHALLENGES AND LEARNING OUTCOMES

14.1 Integration Challenges

Full-stack work requires the frontend and backend to agree on routes, request fields, response fields, and authentication behavior. A login page may look correct but still fail when the payload does not match the DTO or when the frontend reads a different token property from the one returned by the backend. Centralizing API configuration and reviewing network requests helps isolate such issues.

14.2 Environment Challenges

The recorded development process included Maven setup and backend startup failures, database configuration, and repeated backend restarts. These issues demonstrate that successful compilation alone is not enough: environment variables, MySQL availability, compatible JDK versions, port settings, and application properties all influence runtime behavior.

14.3 Team Collaboration

A shared repository allows team members to work independently, but module boundaries and naming conventions must be coordinated. Pulling teammate changes, checking `git status`, reviewing commits, and testing after merges are essential to prevent one member’s changes from breaking another member’s feature.

14.4 Skills Practised

- Java and Spring Boot REST API structure.
- DTO-based request handling and service-layer validation.
- Password hashing and JWT concepts.
- React state/context and protected navigation.
- Axios request/response interceptors.
- MySQL configuration and persistence concepts.
- Git branching, commits, pull requests, and integration debugging.
- Writing testable requirements and documenting verified versus planned functionality.

14.5 Professional Lessons

Security features must be tested, not merely configured. UI restrictions should be supported by backend authorization. Error messages should help users without revealing secrets. Finally, technical documentation is strongest when each claim can be traced to code, a test, or an observed application state.

CHAPTER 15 — FUTURE ENHANCEMENTS AND CONCLUSION

15.1 Future Enhancements

- Complete and verify student/teacher profile retrieval and editing.
- Add automated unit and integration tests for authentication services and protected routes.
- Add refresh-token or re-authentication strategy only if required by the agreed architecture.
- Improve validation messages, loading states, accessibility, and mobile dashboard navigation.
- Complete Academic, Announcements, Clubs/Events, and Chat modules through their respective owners.
- Add institution-level tenant isolation before any multi-college deployment.
- Introduce audit logging for security-relevant events without recording credentials or tokens.
- Deploy through HTTPS with managed secrets, backups, monitoring, and database migration practices.

15.2 Multi-College Readiness

The long-term idea of serving multiple colleges requires more than a college selector. Accounts and records would need a trusted institution identifier, server-side tenant isolation, administrative controls, and tests proving that users from one institution cannot read or modify another institution’s data. This is future scope and should not be described as implemented unless these controls exist.

15.3 Conclusion

CampusConnect provides a practical context for developing a role-aware academic portal. My individual responsibility covers the authentication foundation, student/teacher profile area, and common dashboard shell. The development work brings together React, Axios, Java, Spring Boot, MySQL, password hashing, JWT concepts, and collaborative Git workflows.

The most important outcome is a clearer understanding of how frontend session state, backend identity validation, data persistence, and role-specific navigation depend on one another. The next step is to verify the latest code, execute the documented test cases, capture genuine screenshots, and complete profile behavior before making stronger claims about readiness.

15.4 Final Verification Note

Before printing, update any status labels based on the latest repository, confirm names and enrollment number, insert genuine application screenshots if required by the guide, and obtain approval for the certificate wording.

REFERENCES

26. CampusConnect team repository: https://github.com/Adit841/CampusConnect (project source and documentation; consult the current main/feature branch before submission).
27. React documentation: https://react.dev/
28. Vite documentation: https://vite.dev/guide/
29. React Router documentation: https://reactrouter.com/
30. Axios documentation: https://axios-http.com/docs/intro
31. Spring Boot reference documentation: https://docs.spring.io/spring-boot/
32. Spring Security reference documentation: https://docs.spring.io/spring-security/reference/
33. Spring Data JPA documentation: https://docs.spring.io/spring-data/jpa/reference/
34. MySQL documentation: https://dev.mysql.com/doc/
35. Apache Maven documentation: https://maven.apache.org/guides/
36. JSON Web Token introduction and specification resources: https://jwt.io/introduction and https://www.rfc-editor.org/rfc/rfc7519
37. BCrypt password hashing background: Spring Security Password Storage documentation, https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html

The references above provide technical documentation for the technologies used or discussed. They are not evidence that every optional capability described in those documents is implemented in CampusConnect.

APPENDIX A — DEMONSTRATION CHECKLIST

Check	Action	Done
Environment	Confirm MySQL is running and database configuration is correct.	☐
Backend	Run backend and confirm the expected port is available.	☐
Frontend	Run Vite frontend and verify the API base URL.	☐
Student registration	Create a test student account using valid required fields.	☐
Teacher registration	Create a test teacher account using valid required fields.	☐
Duplicate validation	Attempt to register an existing email and observe the response.	☐
Login	Test valid and invalid credentials.	☐
Token	Inspect safe request metadata; do not expose token in report screenshots.	☐
Dashboard	Verify shell, profile link, navigation, and logout entry point.	☐
Protected API	Test missing/invalid token on a protected endpoint.	☐
Profile	Verify only current user's permitted profile fields are displayed.	☐
Repository	Confirm changes are committed to the intended branch and no secrets are tracked.	☐

Do not tick an item until it has been performed on the final build. Keep screenshots free of passwords, tokens, database secrets, and personal information that is not required for evaluation.

APPENDIX B — SAMPLE API CONTRACT NOTES

B.1 Registration request (illustrative)

The actual request body must be copied from the current RegisterRequest DTO and frontend form. The example below describes the shape only and must not be treated as a guaranteed exact payload:

Field concept	Purpose	Rule to verify
name	Display name	Required/length validation
email	Account email	Valid format and uniqueness
password	User-supplied secret	Minimum policy; never returned
role	STUDENT or TEACHER	Allowlisted values only
enrollmentNo / employeeId	Role-specific identifier	Required for relevant role and unique if schema requires

B.2 Login response

The response shape should be verified against AuthResponse. It may contain a token and a safe user summary such as identity and role. The frontend must use the exact field names returned by the backend. Passwords and password hashes must never be included.

B.3 Error handling

- 400 Bad Request: malformed or invalid fields, where used by the API contract.
- 401 Unauthorized: missing/invalid credentials or authentication token.
- 403 Forbidden: authenticated identity lacks permission for a resource.
- 409 Conflict: duplicate account or identifier, if represented this way by the implementation.
- 500 Internal Server Error: unexpected server fault; the client should not display a stack trace.

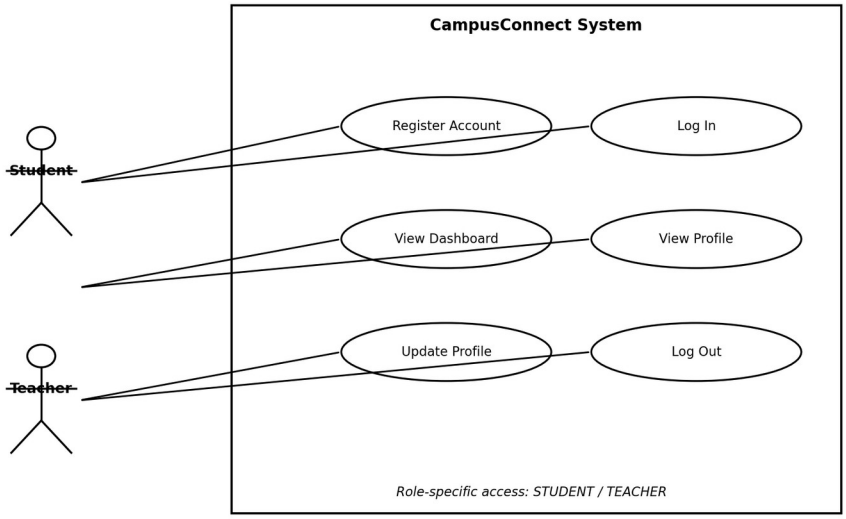
These are recommended contract semantics. Confirm actual status codes and payloads from the current controllers and exception handlers before presenting them as implemented.

UML DIAGRAMS

The following UML diagrams illustrate the individual authentication, user-profile and dashboard scope. They are documentation views based on the intended module design and should be checked against the final code before submission. Where a diagram describes a flow that is not fully verified in the current build, it is labelled accordingly.

Figure A. UML Use Case Diagram

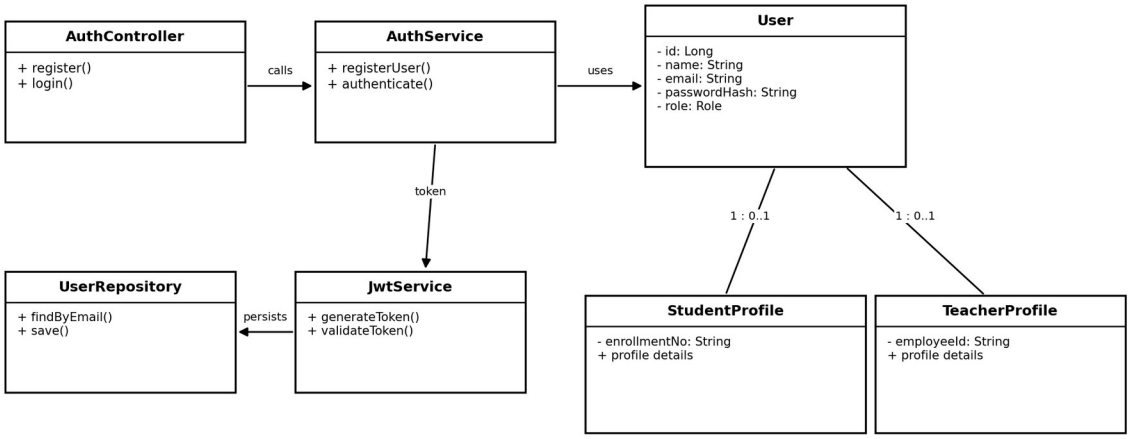
UML Use Case Diagram — CampusConnect Authentication & Profile



Shows the main interactions available to student and teacher users. Profile editing and dashboard access should be confirmed against the running application.

Figure B. UML Class Diagram

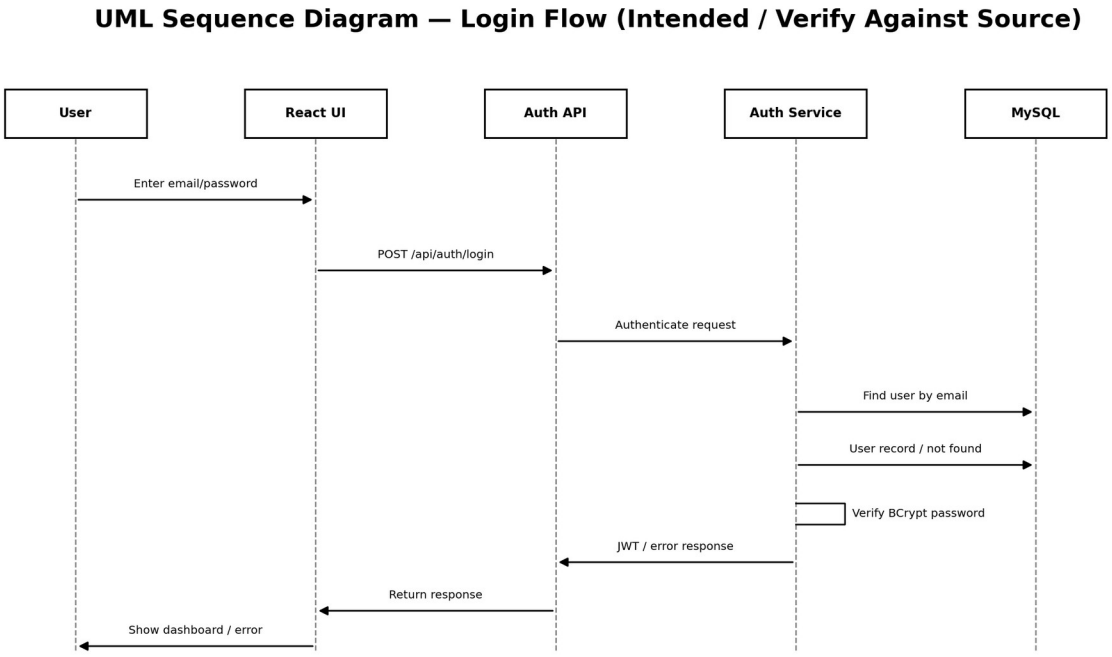
UML Class Diagram — Authentication & User Profile (Conceptual)



Conceptual view; confirm class names and profile relationships against the final source code.

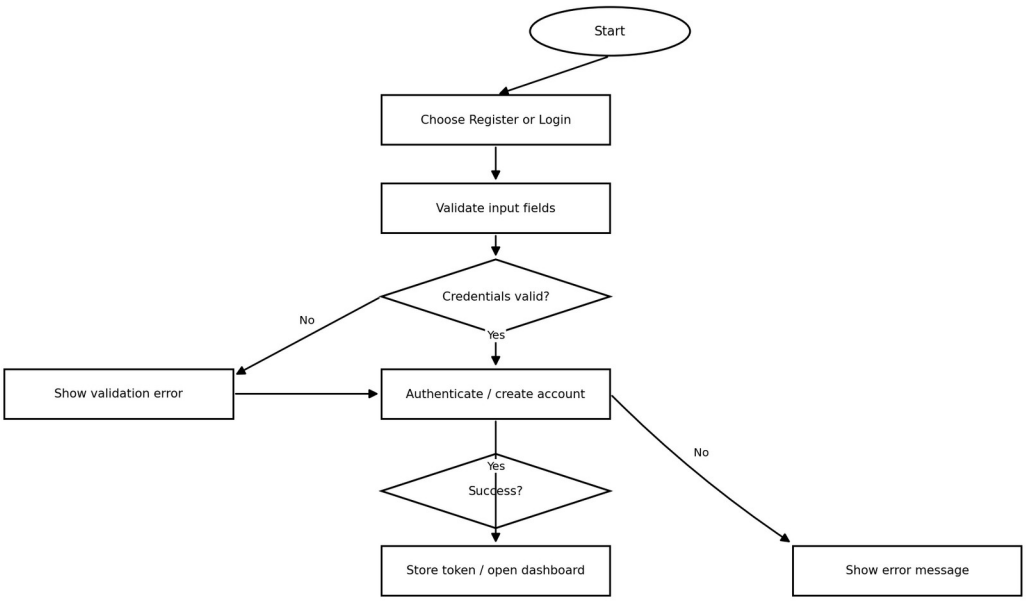
Presents a conceptual class view of controller, service, repository, JWT service, user and role-specific profile data. Confirm exact class/entity names and relationships against source code.

Figure C. UML Sequence Diagram — Login



Illustrates the expected login request path through the React UI, authentication API, service and database. Verify response handling and token storage against the implementation.

Figure D. UML Activity Diagram — Registration/Login



Shows the high-level decision flow for validating input, processing authentication and displaying success or error outcomes.